

 FULL STACK GUIDE

Student Management App

Angular + Spring Boot

A complete beginner-friendly full stack guide — step by step with detailed explanations, code, and everything you need to understand how Angular and Spring Boot work together.

 Spring Boot - Port 8080 Angular - Port 4200 H2 Database - In-Memory

00

How Angular & Spring Boot Work Together

Before writing any code, it is important to understand what role each technology plays and how they communicate. Angular is the **frontend** — the part the user sees and interacts with in the browser. Spring Boot is the **backend** — the engine running on a server that handles logic, processes data, and talks to the database. The two never communicate directly. Instead, they talk through a **REST API** — a standardised way of sending and receiving data over HTTP using a format called **JSON**.

Think of it like a restaurant:

Customer	→	Angular (Frontend)	What the user sees and clicks
Waiter	→	REST API	Carries requests and responses
Kitchen	→	Spring Boot (Backend)	Processes data and talks to DB

When Angular needs data — for example, a list of students — it sends an **HTTP GET request** to Spring Boot's API endpoint. Spring Boot receives it, queries the H2 database, and sends back a **JSON response**. Angular then reads that JSON and displays it on the page. When you add a new student, Angular sends an **HTTP POST request** with the student data as JSON. Spring Boot saves it to the database and returns the saved record (including the generated ID). Angular then adds it to the displayed list immediately — no page reload needed.

API Endpoints used in this app:

GET	<code>/api/students</code>	Returns all students as a JSON array
POST	<code>/api/students</code>	Accepts a student object (JSON) and saves it to the database

01

Project Structure

The project is split into two completely separate applications — the Spring Boot backend and the Angular frontend. Each lives in its own folder and runs independently. They only communicate via HTTP over the network. Here is what each file does:

■ Spring Boot — Backend

`Student.java`

The data model. Defines the shape of a student object with `id`, `name` and `email` fields. Spring Boot uses this to map data to and from the database.

`StudentRepository.java`

Database access layer. Extends `JpaRepository` which automatically provides `findAll()`, `save()`, `delete()` — no SQL needed.

`StudentController.java`

The REST API layer. Defines the `/api/students` endpoints and handles incoming HTTP requests from Angular. Also sets CORS policy.

`application.properties`

Spring Boot configuration file. Sets up the H2 in-memory database connection, driver, and enables the H2 web console.

■ Angular — Frontend

`models/student.ts`

The TypeScript interface. Tells Angular what a student object looks like so it can handle the JSON from Spring Boot correctly.

`services/student.service.ts`

The Angular service that makes HTTP calls to Spring Boot. Contains `getAllStudents()` and `addStudent()` methods using `HttpClient`.

`app.component.ts`

The main Angular component. Contains the logic — loading students on start, handling form submission, and managing state.

`app.component.html`

The HTML template. Contains the add-student form and the student table. Uses Angular directives like `*ngFor` and `[(ngModel)]`.

02

Spring Boot — Backend

STEP 01

Create the Project via Spring Initializr

<https://start.spring.io>

Spring Initializr is an official web tool provided by the Spring team that generates a ready-to-use Spring Boot project. Instead of setting up the folder structure, Maven configuration, and dependencies manually, you configure it on the website and it generates everything for you as a zip file. This saves a significant amount of setup time and ensures the project structure is correct from the start.

Field	Value	Why
Project	Maven	Maven is the build tool — it downloads dependencies and compiles your code
Language	Java	The language Spring Boot is written in
Spring Boot	3.x.x (latest)	Always use the latest stable version
Group	com.example	Like a package name — identifies your organisation
Artifact	students	The name of your application
Packaging	Jar	Packages the app as a single runnable file
Java	17 or 21	LTS versions — stable, long-term supported releases

Dependencies to add: Search and add **Spring Web** (enables REST API and web server), **Spring Data JPA** (database access layer using Java Persistence API), and **H2 Database** (an in-memory database — no installation needed, perfect for learning).

After generating, move the project into WSL:

bash

```
# After downloading and extracting the zip file
mv /mnt/c/Users/YourName/Downloads/students ~/students
cd ~/students
```

STEP 02

Student.java — The Data Model

```
src/main/java/com/example/students/Student.java
```

The **Student** class is the data model — it defines what a student looks like in your application. Spring Boot uses this class to automatically create a database table called **STUDENT** with columns for id, name, and email. This is done through **JPA (Java Persistence API)** annotations:

Annotation	What it does
<code>@Entity</code>	Tells Spring Boot this class maps to a database table
<code>@Id</code>	Marks the id field as the primary key of the table
<code>@GeneratedValue</code>	Makes the database auto-generate the ID (1, 2, 3...) — you never set it manually

```
java
```

```
package com.example.students;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;

    public Student() {}

    public Student(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public void setId(Long id) { this.id = id; }
    public void setName(String name) { this.name = name; }
    public void setEmail(String email) { this.email = email; }
}
```

STEP 03

StudentRepository.java — Database Access

```
src/main/java/com/example/students/StudentRepository.java
```

The repository is the layer that communicates with the database. Rather than writing SQL queries manually, Spring Data JPA provides a powerful interface called **JpaRepository**. By simply extending it and specifying your entity type (**Student**) and the ID type (**Long**), you get a full set of database operations for free — with zero implementation code needed from you.

Methods you get for free: `findAll()` — get all records, `save(entity)` — insert or update, `findById(id)` — get one record, `deleteById(id)` — remove a record, `count()` — total records, and many more. Spring generates the actual SQL behind the scenes.

```
java
```

```
package com.example.students;

import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

STEP 04

StudentController.java — REST API Handler

```
src/main/java/com/example/students/StudentController.java
```

The controller is the most important class for the API. It defines the HTTP endpoints that Angular calls. Each method in the controller is mapped to a specific HTTP method (GET, POST etc.) and URL path. Spring Boot automatically converts the returned Java objects into JSON and sends them back to Angular.

Annotation	What it does
<code>@RestController</code>	Marks this class as a REST API controller. Every method returns data (JSON), not a webpage.
<code>@RequestMapping("/api/students")</code>	All endpoints in this class start with <code>/api/students</code> as the base URL path.
<code>@CrossOrigin(origins = "http://localhost:4200")</code>	Allows Angular (running on port 4200) to call this API. Without this, browsers block cross-origin requests (CORS policy).
<code>@GetMapping</code>	Maps the <code>getAllStudents()</code> method to HTTP GET requests on <code>/api/students</code> .
<code>@PostMapping</code>	Maps the <code>addStudent()</code> method to HTTP POST requests on <code>/api/students</code> .
<code>@RequestBody</code>	Tells Spring to read the JSON body of the POST request and convert it into a <code>Student</code> object.

java

```
package com.example.students;

import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/students")
@CrossOrigin(origins = http://localhost:4200)
public class StudentController {

    private final StudentRepository repository;

    public StudentController(StudentRepository repository) {
        this.repository = repository;
    }

    @GetMapping
    public List<Student> getAllStudents() {
        return repository.findAll();
    }

    @PostMapping
    public Student addStudent(@RequestBody Student student) {
        return repository.save(student);
    }
}
```

STEP 05

application.properties — H2 Database Configuration

src/main/resources/application.properties

This file is Spring Boot's main configuration file. It uses a simple key=value format. Here you configure the H2 in-memory database connection. H2 is a lightweight Java database that runs entirely inside your application's memory — no separate database server installation required. It is perfect for learning and development because it starts automatically with your application and requires zero setup.

Important: Since H2 is in-memory, all data is lost when Spring Boot stops. This is intentional for learning — when you are ready for production, you would replace these settings with MySQL or PostgreSQL connection details.

properties

```
spring.datasource.url=jdbc:h2:mem:studentsdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

STEP 06

Run the Spring Boot Backend

Navigate to the project folder and use the Maven Wrapper script to start the application. **mvnw** is a script included in the generated project — it downloads the correct version of Maven automatically if needed, so you don't have to install Maven separately. The **spring-boot:run** command compiles and starts the embedded Tomcat web server.

```
bash
```

```
cd ~/students
./mvnw spring-boot:run

# You should see these lines in the output:
Tomcat started on port 8080
Started StudentsApplication in 2.4 seconds

# Test it in your Windows browser:
http://localhost:8080/api/students
# Expected: [] ← empty array means it is working
```

03

Angular — Frontend

STEP 01

Create the Angular Project

Open a **second WSL terminal** window — keep Spring Boot running in the first one. The Angular CLI **ng new** command generates a complete Angular project with the correct folder structure, TypeScript configuration, and all required packages. It will ask you two questions during setup:

Question	Answer	Why
Which stylesheet format?	CSS	Plain CSS is the simplest — no preprocessor needed for a beginner project
Enable Server-Side Rendering?	N	SSR adds complexity we don't need — client-side rendering is fine for this app

bash

```
cd ~
ng new student-app

# Answer the prompts:
# Which stylesheet format? → CSS
# Enable Server-Side Rendering? → N

cd student-app
```

STEP 02

student.ts — The Data Interface

src/app/models/student.ts

A TypeScript **interface** defines the shape of an object — it tells Angular what properties a student has and what type each property is. This is purely a TypeScript concept for type safety; it does not generate any JavaScript at runtime. It ensures that when Angular receives JSON from Spring Boot, TypeScript knows the structure and can catch errors during development if you try to access a property that doesn't exist.

Why id? — The question mark after id makes it optional. When you create a new student to send to Spring Boot, you don't have an ID yet — Spring Boot generates it. But when Spring Boot returns the saved student, it includes the ID. The ? lets Angular handle both cases without TypeScript errors.

typescript

```
export interface Student {
  id? : number; // optional – generated by Spring Boot
  name : string; // required
  email: string; // required
}
```

STEP 03

student.service.ts — API Communication Layer

src/app/services/student.service.ts

The service is responsible for all communication with the Spring Boot backend. It uses Angular's built-in **HttpClient** to make HTTP requests. By isolating API calls in a service, the component doesn't need to know anything about HTTP — it just calls service methods and gets data back. The methods return **Observables** — Angular's way of handling asynchronous operations (operations that take time, like network requests). The component subscribes to these observables and reacts when the data arrives.

typescript

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Student } from '../models/student';

@Injectable({ providedIn: 'root' })
export class StudentService {

  private apiUrl = 'http://localhost:8080/api/students';

  constructor(private http: HttpClient) {}

  getAllStudents(): Observable<Student[]> {
    return this.http.get<Student[]>(this.apiUrl);
  }

  addStudent(student: Student): Observable<Student> {
    return this.http.post<Student>(this.apiUrl, student);
  }
}
```

STEP 04

app.config.ts — Register HttpClient

src/app/app.config.ts

Angular uses a providers system to make services available throughout the app. **provideHttpClient()** registers Angular's HTTP module so that HttpClient can be injected into your service. Without this line, Angular would throw an error when your service tries to use HttpClient because it wouldn't be available. This is the modern way to configure Angular (standalone components, introduced in Angular 17+).

typescript

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient() // ← this is the important line
  ]
};
```

STEP 05

app.component.ts — Component Logic

src/app/app.component.ts

The component is the brain of the Angular UI. It holds the **state** (the list of students, the form values, messages) and the **logic** (what happens when the page loads, what happens when you click Add Student). The **ngOnInit()** lifecycle hook runs automatically when the component is created — this is where you load initial data. The **subscribe()** calls wait for the Observable to complete and then run the callback functions with the result.

typescript

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { StudentService } from '../services/student.service';
import { Student } from '../models/student';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent implements OnInit {

  students: Student[] = [];
  newStudent: Student = { name: '', email: '' };
  successMessage = '';

  constructor(private studentService: StudentService) {}

  ngOnInit(): void {
    this.loadStudents(); // runs automatically on page load
  }

  loadStudents(): void {
    this.studentService.getAllStudents().subscribe({
      next: (data) => this.students = data,
      error: (err) => console.error('Error loading' , err)
    });
  }

  addStudent(): void {
    if (!this.newStudent.name || !this.newStudent.email) return;

    this.studentService.addStudent(this.newStudent).subscribe({
      next: (student) => {
        this.students.push(student);
        this.newStudent = { name: '', email: '' };
        this.successMessage = 'Student added successfully!' ;
        setTimeout(() => this.successMessage = '', 3000);
      },
      error: (err) => console.error('Error adding' , err)
    });
  }
}
```

STEP 06

app.component.html — UI Template

```
src/app/app.component.html
```

The HTML template defines what the user sees. Angular enhances standard HTML with **directives** — special attributes that add behaviour. **[(ngModel)]** is two-way data binding — when the user types in the input, the component property updates automatically, and if the property changes in code, the input updates too. ***ngIf** conditionally shows or hides elements based on a boolean condition. ***ngFor** loops over an array and renders one element per item. **(click)** binds a button click to a component method.

```
html
```

```
<div class="container">
<h1>Student Management</h1>

<!-- Add Student Form -->
<div class="form-card">
<h2>Add New Student</h2>

<input type="text"
placeholder="Student Name"
[(ngModel)]="newStudent.name" />

<input type="email"
placeholder="Student Email"
[(ngModel)]="newStudent.email" />

<button (click)="addStudent()">Add Student</button>
<p *ngIf="successMessage">{{ successMessage }}</p>
</div>

<!-- Student List -->
<div class="list-card">
<h2>Students ({{ students.length }})</h2>

<table *ngIf="students.length > 0">
<thead><tr><th>ID</th><th>Name</th><th>Email</th></tr></thead>
<tbody>
<tr *ngFor="let student of students">
<td>{{ student.id }}</td>
<td>{{ student.name }}</td>
<td>{{ student.email }}</td>
</tr>
</tbody>
</table>
</div>
</div>
```

STEP 07

app.component.css — Styling

src/app/app.component.css

The CSS file styles the component. Angular applies these styles only to this component (component-scoped CSS) so they don't accidentally affect other parts of the app. The styling is intentionally kept simple — a centred container, card-style boxes with shadows, clean inputs, and a blue button — to keep the focus on learning the full stack concepts rather than complex UI design.

CSS

```
.container {
max-width: 800px; margin: 40px auto;
padding: 0 20px; font-family: Arial, sans-serif;
}
.form-card, .list-card {
background: #fff; border-radius: 8px;
padding: 24px; margin-bottom: 24px;
box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
input {
display: block; width: 100%; padding: 10px;
border: 1px solid #ddd; border-radius: 4px;
}
button {
background: #1976d2; color: white;
padding: 10px 24px; border-radius: 4px;
}
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border-bottom: 1px solid #eee; }
th { background: #f5f5f5; }
```

STEP 08

Run Angular

With Spring Boot still running in Terminal 1, run Angular in Terminal 2. The **ng serve** command compiles the TypeScript, bundles the app, and starts a development server with **live reload** — any time you save a file, the browser automatically updates. Open your Windows browser and navigate to the app:

bash

```
ng serve

# You should see:
✓ Compiled successfully
Application bundle generation complete.

# Open in Windows browser:
http://localhost:4200
```

04

What Happens Behind the Scenes

Understanding the flow of data is essential to understanding full stack development. Here is exactly what happens step by step for each user action:

When the Page Loads

1 Angular starts and calls `ngOnInit()`

The `ngOnInit()` lifecycle hook fires as soon as the `AppComponent` is created. This is Angular's way of saying 'the component is ready — do your initialisation here'.

2 Angular calls `getAllStudents()` on the service

The component tells the `StudentService` to fetch data. The service uses `HttpClient` to build a GET request to `http://localhost:8080/api/students`.

3 Spring Boot receives the GET `/api/students` request

The `StudentController`'s `getAllStudents()` method handles the request. It calls `repository.findAll()` which generates `SELECT * FROM STUDENT` and runs it against H2.

4 Spring Boot returns a JSON array

The list of `Student` objects is automatically serialised into JSON by Spring Boot and sent back as the HTTP response body.

5 Angular receives the JSON and updates the view

The `subscribe()` callback receives the student array, assigns it to `this.students`, and Angular's `*ngFor` directive re-renders the table rows automatically.

When You Add a Student

1 User fills the form and clicks Add Student

The inputs are bound to `newStudent.name` and `newStudent.email` via `[(ngModel)]`. Clicking the button triggers the `addStudent()` method in the component.

2 Component validates and calls `addStudent()` on the service

The method first checks that name and email are not empty. If valid, it passes the `newStudent` object to `StudentService.addStudent()`.

3 Angular sends a POST request to /api/students

HttpClient serialises the Student object into JSON and sends it as the request body to `http://localhost:8080/api/students` with `Content-Type: application/json`.

4 Spring Boot receives, saves, and returns the student

`@RequestBody` deserialises the JSON back into a Student object. `repository.save()` runs `INSERT INTO STUDENT` and H2 assigns the next available ID automatically.

5 Angular adds the new student to the table instantly

The `subscribe()` callback receives the saved student (now with an ID), pushes it into the `students` array, clears the form, and shows a success message for 3 seconds.

05

Important Notes

H2 IN-MEMORY DATABASE

Since H2 runs entirely in memory, all student data is lost the moment Spring Boot stops. This is expected and fine for learning. When you are ready to build a real application, you would replace the H2 configuration with MySQL or PostgreSQL settings and your data would persist across application restarts.

@CROSSORIGIN AND CORS

CORS (Cross-Origin Resource Sharing) is a browser security policy that blocks requests between different origins. Angular on port 4200 and Spring Boot on port 8080 are considered different origins. The `@CrossOrigin` annotation tells Spring Boot to include special headers in its responses that tell the browser to allow these cross-origin requests.

PORTS

Spring Boot always runs on port 8080 and Angular always runs on port 4200. Both terminals must be running simultaneously for the app to work. If either port is already in use by another process, the server will fail to start and you will need to either stop the conflicting process or change the port.

JPAREPOSITORY MAGIC

When you extend `JpaRepository`, Spring Data JPA generates an implementation class at runtime with all the standard database methods already coded. You only need to write custom query methods if you have non-standard requirements — for example, finding students by name would be as simple as adding `findByName(String name)` to the interface.

06

Suggested Next Steps

Now that your first full stack application is running, here are natural next steps to grow your skills and expand the project:

Delete a Student

Add a `DELETE /api/students/{id}` endpoint to Spring Boot with `@DeleteMapping`. Then add a delete button to each table row in Angular that calls the endpoint via the service. This teaches you path variables (`@PathVariable`) and HTTP `DELETE` method.

Edit a Student

Add a `PUT /api/students/{id}` endpoint with `@PutMapping` to Spring Boot. In Angular, add an edit button that populates a form with the student's current data. This teaches you HTTP `PUT`, two-way data binding, and conditional UI rendering.

Add Validation

Use Spring Boot's `@Valid` annotation and Jakarta Validation annotations like `@NotBlank` and `@Email` on the `Student` model. In Angular, use reactive forms with `Validators`. This teaches input validation on both frontend and backend.

Switch to MySQL

Replace H2 with MySQL for persistent storage. Update `application.properties` with the MySQL JDBC URL and credentials. Add the MySQL connector dependency to `pom.xml`. This teaches production database configuration and persistence.

Angular Routing

Use Angular Router to create multiple pages — a student list page, a student detail page, and a dedicated add student page. This teaches single-page application navigation and component-based architecture.